API Reference

The versioning rule, authentication, the shared response envelopes, DTO conventions, and the full endpoint reference.

The Finance Portal exposes a versioned REST API behind the Nginx gateway. This document is the contract: the versioning rule, how authentication works, the response envelopes every endpoint shares, the DTO conventions, and the full endpoint map. Errors have their own document — see *Error Handling*.

Versioning

Every endpoint is served under a URI version prefix:

```
/api/v1/<resource>
```

- The prefix (`v1`) is the **major** API version. Breaking changes — a removed field, a changed type, a different status code — ship under a new prefix (`/api/v2`) so existing clients keep working. Additive, backward-compatible changes stay in `v1` .
- The product itself is released with semantic versioning (`v0.x` during development); the API prefix tracks only the wire contract, independent of the product version.
- The gateway routes `/api/v1/{notifications,notification-preferences,price-alerts,watchlists, reports,market-status,...}` to the notification service (`:8082`) and everything else under `/api/v1` to the core backend (`:8080`). Clients see one consistent base path.

Interactive documentation (OpenAPI / Swagger)

The API is described by an OpenAPI 3.1 spec generated from the controllers (Springdoc):

- **Swagger UI** (interactive, try-it-out): `http://localhost/swagger-ui/index.html`
- **OpenAPI spec** (machine-readable): `http://localhost/v3/api-docs` — a snapshot is kept at `docs/api/openapi.json` .

Each endpoint's **full contract — path & query parameters, request-body fields, validation rules, response schema and status codes — lives there**, generated from the controllers so it never drifts from the code. The endpoint map at the end of this document is the orientation layer; Swagger UI / `openapi.json` is the authoritative parameter reference.

Recurring query parameters: list endpoints page with `page` (0-indexed) and `size` , and many accept a `sort` / type / search filter — the exact set per resource is in Swagger.

Authentication

The API is stateless and secured with **Keycloak (OIDC)**. Every request (except the two public endpoints noted below) carries a bearer token:

```
Authorization: Bearer <access_token>
```

- The token is a Keycloak JWT; the backend validates it against the realm's JWKS.
- The caller is identified by the token's `sub` claim — all user-scoped data (portfolios, alerts, preferences) is keyed on it.
- Admin endpoints additionally require the realm role `ADMIN` (`@PreAuthorize`).
- Public endpoints: `GET /api/v1/markets/funds/sub-categories` and `GET /api/v1/notifications/ping`.

The gateway also rate-limits: ~50 req/s for general traffic, ~5 req/s on auth-sensitive routes.

Response envelope — `ApiResponse<T>`

Every successful response is wrapped in one envelope, so clients parse a single shape. Controllers never return entities directly.

```
public class ApiResponse<T> {
    private boolean success;        // true
    private String message;         // optional, localized
    private T data;                 // payload: a DTO, a PagedResponse<T>, a collection, or a
scalar
    private LocalDateTime timestamp;
}
```

```
{
  "success": true,
  "message": "Portfolio created",
  "data": { "id": 12, "name": "Ana Portföy", "createdAt": "2026-05-29T10:15:30" },
  "timestamp": "2026-05-29T10:15:30"
}
```

Pagination — `PagedResponse<T>`

List endpoints wrap their items in a `PagedResponse<T>`, nested inside `ApiResponse.data`:

```
public record PagedResponse<T>(
    List<T> content, int page, int size, long totalElements, int totalPages) {}
```

```
{
  "success": true,
  "data": { "content": [ /* … */ ], "page": 0, "size": 20, "totalElements": 137, "totalPages": 7 },
  "timestamp": "2026-05-29T10:15:30"
}
```

Errors — `ErrorResponse`

Failures use a parallel `ErrorResponse` envelope (`success: false`, `errorCode`, `message`, `path`, …) so success and error paths are handled uniformly on the client. The full field list and the exception → HTTP status map are in the **Error Handling** document.

DTO & mapping conventions

```
com.finance.{module}.dto
├── request/    # input DTOs — suffix *Request, bean-validation annotated
└── response/   # output DTOs — suffix *Response
```

- DTOs are immutable Java **records**; entities never cross the controller boundary.
- Request DTOs validate declaratively (`@NotNull`, `@NotBlank`, `@Positive`); a violation becomes a `400 VALIDATION_ERROR` with a per-field map.
- No `Map<String, Object>` on the response surface — type-specific data uses **sealed** records (e.g. `MarketAssetMetadata` permits `StockMetadata`, `FundMetadata`, …).
- Entity → DTO mapping is done with **MapStruct** (`@Mapper(componentModel = "spring")`); computed fields (live price, P&L) are passed in by the service. `@AfterMapping` hooks enforce monetary scaling (`scale = 4` money, `8` quantity).

```
public record PositionRequest(
    @NotBlank String assetType, @NotBlank String assetCode,
    @NotNull @Positive BigDecimal quantity,
    @NotNull LocalDateTime entryDate, @NotNull @Positive BigDecimal entryPrice,
    LocalDateTime exitDate, @Positive BigDecimal exitPrice,
    String priceCurrency /* ISO-4217; null = already TRY */) {}
```

Endpoint map

All paths are relative to `/api/v1`. Authenticated unless marked **Public**; **Admin** needs the `ADMIN` role.

User & profile

METHOD	PATH	PURPOSE
GET / PUT	`/user/profile`	Get / update the current user's profile
POST	`/user/credentials/password/initiate-change`	Start password reset (Keycloak email)
POST	`/user/credentials/email/initiate-change` · `/confirm-change`	Email change flow
GET / DELETE	`/user/credentials/2fa` · `/2fa/devices/{id}`	2FA status / disable / remove device
GET / PATCH	`/user/preferences`	Read / partially update preferences

METHOD	PATH	PURPOSE
GET / PUT	/user/layout · /user/layout/overview	Dashboard layout
GET / PUT	/user/chart-data/{type}/{code}/…	Per-asset chart prefs + drawings

Portfolios & positions

METHOD	PATH	PURPOSE
GET / POST	/portfolios	List / create portfolios
PUT / DELETE	/portfolios/{id}	Rename / delete
GET	/portfolios/{id}/view	Combined summary + positions + allocation
GET	/portfolios/{id}/summary · /allocation	Totals & returns · allocation breakdown
GET / POST	/portfolios/{id}/positions	List (paged) / add a spot position
PUT / DELETE	/portfolios/{id}/positions/{pid}	Update / delete a position
POST	/portfolios/{id}/positions/{pid}/sell · /reopen	Close / reopen
GET	/portfolios/{id}/backfill-stream	Snapshot backfill progress (SSE)
GET / POST	/portfolios/{id}/derivatives	VIOP derivative positions
PATCH / PUT	/portfolios/{id}/derivatives/{pid}/close · /reopen	Close / amend / reopen

Market, instruments & analytics

METHOD	PATH	PURPOSE
GET	/market	Unified cross-asset search (paged)
GET	/market/history · /group-counts · /availability	History · counts · coverage
GET	/market/overview · /overview/widget-definitions	Rendered widgets · catalog
GET	/market-status	Open/closed session per market
GET	/bonds · /bonds/{seriesCode} · /bonds/rate-history/{isin}	Bonds + rate history
GET	/macro-indicators · /{code}/history	Macro indicators
GET	/bank-rates · /bank-rates/currencies	Exchange / asset rates
GET	/tracked-assets · /{type}/{code}	Tracked-asset registry
POST	/analytics/scenarios	Scenario simulation
GET	/analytics/inflation-beaters · /portfolio-series/{id}	Inflation ranking · value series

News & reports

METHOD	PATH	PURPOSE
GET	`/news` · `/news/categories` · `/news/{id}`	List / categories / detail
POST	`/reports/portfolio-pdf`	Generate a portfolio PDF report

Notifications, alerts & watchlists *(notification service, :8082)*

METHOD	PATH	PURPOSE
GET	`/notifications` · `/unread-count` · `/stream` (SSE)	List / count / live stream
PATCH / POST / DELETE	`/notifications/{id}/read` · `/read-all` · `/{id}`	Read / bulk read / delete
GET / PATCH	`/notification-preferences`	Read / update preferences
POST / GET / PUT / DELETE	`/price-alerts` …	Manage price alerts
GET / POST / PUT / DELETE	`/watchlists` · `/watchlists/{id}/items` …	Manage watchlists

Admin

METHOD	PATH	PURPOSE
GET / PUT	`/admin/users` · `/{id}/ban` · `/unban`	Keycloak user management
GET / POST / PATCH / DELETE	`/admin/tracked-assets` · `/admin/news-sources`	Reference data management
POST	`/admin/trigger/{type}/{snapshot|candles|full}`	Trigger a data refresh (202)
GET	`/admin/tasks/stream`	Live task status (SSE)
POST	`/admin/notifications/broadcast`	Broadcast a system notification

> The exhaustive, always-current list (request/response schemas, every field) lives in Swagger UI and `openapi.json` — this map is the orientation layer on top of it.

Parameters reference

Generated from the OpenAPI spec. `*` = required. Path params appear in the path; **Body** names a request DTO (fields are in the DTO section / Swagger).

admin-controller

ENDPOINT	QUERY PARAMS	BODY
`POST /admin/trigger/{type}/snapshot`	—	—
`POST /admin/trigger/{type}/full`	—	—

ENDPOINT	QUERY PARAMS	BODY
`POST /admin/trigger/{type}/candles`	—	—
`POST /admin/trigger/news/update`	—	—
`POST /admin/trigger/macro/refresh`	—	—
`POST /admin/trigger/bond/update`	—	—
`GET /admin/tasks/stream`	—	—

admin-news-source-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /admin/news-sources/{id}`	—	—
`PUT /admin/news-sources/{id}`	—	`UpsertNewsSourceRequest`
`DELETE /admin/news-sources/{id}`	—	—
`GET /admin/news-sources`	includeDisabled (boolean)	—
`POST /admin/news-sources`	—	`UpsertNewsSourceRequest`
`PATCH /admin/news-sources/{id}/enabled`	enabled* (boolean)	—

admin-tracked-asset-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /admin/tracked-assets`	type (string), search (string), sort (string), direction (string)	—
`POST /admin/tracked-assets`	—	`UpsertTrackedAssetRequest`
`PATCH /admin/tracked-assets/order`	—	`BulkTrackedAssetOrderUpdateRequest`
`DELETE /admin/tracked-assets/{type}/{code}`	—	—

admin-user-controller

ENDPOINT	QUERY PARAMS	BODY
`PUT /admin/users/{id}/unban`	—	—
`PUT /admin/users/{id}/ban`	—	—
`GET /admin/users`	first (integer), max (integer), search (string)	—
`GET /admin/users/count`	search (string)	—

analytics-controller

ENDPOINT	QUERY PARAMS	BODY
`POST /analytics/scenarios`	—	`ScenarioRequest`
`GET /analytics/portfolio-series/{portfolioId}`	from* (string), to* (string)	—
`GET /analytics/inflation-beaters`	period (string), benchmark (string), targetCurrency (string)	—

bank-rates-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /bank-rates`	currency (string), kind (string)	—
`GET /bank-rates/currencies`	kind (string)	—

bond-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /bonds`	search (string), bondType (string), sort (string), direction (string), page (integer), size (integer)	—
`GET /bonds/{seriesCode}`	—	—
`GET /bonds/types`	—	—
`GET /bonds/rate-history/{isinCode}`	period (string)	—

derivative-position-controller

ENDPOINT	QUERY PARAMS	BODY
`PUT /portfolios/{portfolioId}/derivatives/{positionId}`	—	`UpdateDerivativePositionRequest`
`DELETE /portfolios/{portfolioId}/derivatives/{positionId}`	—	—
`PUT /portfolios/{portfolioId}/derivatives/{positionId}/close`	—	`CloseDerivativePositionRequest`
`PATCH /portfolios/{portfolioId}/derivatives/{positionId}/close`	—	`CloseDerivativePositionRequest`
`GET /portfolios/{portfolioId}/derivatives`	openOnly (boolean)	—
`POST /portfolios/{portfolioId}/derivatives`	—	`OpenDerivativePositionRequest`
`PATCH /portfolios/{portfolioId}/derivatives/{positionId}/reopen`	—	—

fund-detail-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /markets/funds/sub-categories`	—	—

macro-indicator-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /macro-indicators`	category (string), prominentOnly (boolean)	—
`GET /macro-indicators/{code}/history`	from (string), to (string)	—

market-overview-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /market/overview`	page (string)	—
`GET /market/overview/widget-definitions`	—	—

news-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /news`	category (string), search (string), sort (string), direction (string), page (integer), size (integer)	—
`GET /news/{id}`	—	—
`GET /news/categories`	—	—

portfolio-controller

ENDPOINT	QUERY PARAMS	BODY
`PUT /portfolios/{portfolioId}`	—	`PortfolioCreateRequest`
`DELETE /portfolios/{portfolioId}`	—	—
`PUT /portfolios/{portfolioId}/positions/{positionId}`	—	`PositionRequest`
`DELETE /portfolios/{portfolioId}/positions/{positionId}`	—	—
`GET /portfolios`	—	—
`POST /portfolios`	—	`PortfolioCreateRequest`

ENDPOINT	QUERY PARAMS	BODY
`GET /portfolios/{portfolioId}/positions`	search (string), assetType (string), sort (string), direction (string), page (integer), size (integer)	—
`POST /portfolios/{portfolioId}/positions`	—	`PositionRequest`
`POST /portfolios/{portfolioId}/positions/{positionId}/sell`	—	`PositionSellRequest`
`POST /portfolios/{portfolioId}/positions/{positionId}/reopen`	—	—
`GET /portfolios/{portfolioId}/view`	include (string)	—
`GET /portfolios/{portfolioId}/summary`	assetType (string)	—
`GET /portfolios/{portfolioId}/chart`	type* (string), range (string), assetType (string), assetCode (string)	—
`GET /portfolios/{portfolioId}/backfill-stream`	—	—
`GET /portfolios/{portfolioId}/assets/{assetType}/{assetCode}/summary`	—	—
`GET /portfolios/{portfolioId}/allocation`	mode (string), assetType (string), limit (integer)	—
`GET /portfolios/limits`	—	—

tracked-asset-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /tracked-assets`	type (string), search (string), sort (string), direction (string)	—

ENDPOINT	QUERY PARAMS	BODY
`GET /tracked-assets/{type}/{code}`	—	—

unified-market-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /market`	type (string), code (string), segment (string), search (string), subType (string), sort (string), direction (string), filter (string), page (integer), size (integer), subCategory (array), riskValue (array)	—
`GET /market/history`	type* (string), code* (string), period (string)	—
`GET /market/group-counts`	type* (string)	—
`GET /market/availability`	type* (string), code* (string), yearMonth* (string)	—

user-chart-data-controller

ENDPOINT	QUERY PARAMS	BODY
`PUT /user/chart-data/{type}/{code}/preferences`	—	`UserChartPreferenceUpdateRequest`
`PUT /user/chart-data/{type}/{code}/drawings`	—	`UserChartDrawingUpdateRequest`
`GET /user/chart-data/{type}/{code}`	range (string)	—

user-credential-controller

ENDPOINT	QUERY PARAMS	BODY
`POST /user/credentials/password/initiate-change`	—	`PasswordChangeInitiateRequest`
`POST /user/credentials/email/initiate-change`	—	`EmailChangeInitiateRequest`
`POST /user/credentials/email/confirm-change`	—	`EmailChangeConfirmRequest`
`GET /user/credentials/email/pending`	—	—
`DELETE /user/credentials/email/pending`	—	—
`GET /user/credentials/2fa`	—	—
`DELETE /user/credentials/2fa`	—	—
`GET /user/credentials/2fa/devices`	—	—
`DELETE /user/credentials/2fa/devices/{credentialId}`	—	—

user-layout-controller

ENDPOINT	QUERY PARAMS	BODY
`PUT /user/layout/overview`	—	`JsonNode`

ENDPOINT	QUERY PARAMS	BODY
`GET /user/layout`	—	—

user-preference-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /user/preferences`	—	—
`PATCH /user/preferences`	—	`UserPreferenceUpdateRequest`

user-profile-controller

ENDPOINT	QUERY PARAMS	BODY
`GET /user/profile`	—	—
`PUT /user/profile`	—	`ProfileUpdateRequest`